

LAB 4

TITLE: Implementation of Round Robin (RR) CPU Scheduling Algorithm in C

OBJECTIVE:

To implement **Round Robin Scheduling** algorithm using C, and calculate:

- **Waiting Time (WT)**
- **Turnaround Time (TAT)**
- **Average Waiting Time**
- **Average Turnaround Time**

THEORY:

Round Robin (RR) is a **preemptive** CPU scheduling algorithm that assigns a fixed time quantum to each process in the ready queue. Processes are executed in a circular (round-robin) order.

➤ **Characteristics:**

- Every process gets equal share of CPU time (quantum).
- If a process doesn't finish in its time quantum, it is sent to the end of the queue.
- Fair and starvation-free scheduling.
- Preemptive in nature.

➤ **Terminologies:**

- **Burst Time (BT):** Total time required by a process to complete.
- **Time Quantum:** The fixed time allocated to each process in each cycle.
- **Waiting Time (WT):** Total time a process waits in the ready queue.
- **Turnaround Time (TAT):** Total time from arrival to completion.
 - $TAT[i] = BT[i] + WT[i]$
- **Average Waiting/Turnaround Time:**
 - $AWT = \sum WT / n$, $ATAT = \sum TAT / n$

Algorithm:

1. Input number of processes n and time quantum q
2. Input burst time for each process
3. Initialize remaining burst time = burst time
4. Loop through processes in order:
 - If remaining time $>$ quantum:
 - Execute for q time
 - Subtract quantum from remaining
 - Else:
 - Execute remaining time
 - Set waiting time = current time - original burst time
 - Mark process as complete
5. After all processes are complete, calculate turnaround time:
 - $TAT[i] = BT[i] + WT[i]$
6. Compute average WT and TAT
7. Display result

PROGRAMS

```
#include <stdio.h>
```

```
int main() {
    int n, quantum;
    printf("Enter total number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    int burst_time[n], waiting_time[n],
    turnaround_time[n], remaining_time[n];
    int i, time = 0, completed = 0;

    // Input burst times
    for(i = 0; i < n; i++) {
        printf("Enter burst time for process P%d: ", i
+ 1);
        scanf("%d", &burst_time[i]);
        remaining_time[i] = burst_time[i];
        waiting_time[i] = 0;
    }

    // Round Robin scheduling
    while(completed < n) {
        for(i = 0; i < n; i++) {
            if(remaining_time[i] > 0) {
                if(remaining_time[i] > quantum) {
                    time += quantum;
                    remaining_time[i] -= quantum;
                } else {
                    time += remaining_time[i];
                    waiting_time[i] = time -
burst_time[i];
                    remaining_time[i] = 0;
                    completed++;
                }
            }
        }

        // Calculate turnaround times
        float avg_waiting = 0, avg_turnaround = 0;
        for(i = 0; i < n; i++) {
            turnaround_time[i] = burst_time[i] +
waiting_time[i];
```

```
            avg_waiting += waiting_time[i];
            avg_turnaround += turnaround_time[i];
        }

        avg_waiting /= n;
        avg_turnaround /= n;

        // Display results
        printf("\nProcess\t\tBurst Time\tWaiting
Time\tTurnaround Time\n");
        for(i = 0; i < n; i++) {
            printf("P%d\t\t%d\t\t%d\t\t%d\n", i+1,
burst_time[i], waiting_time[i],
turnaround_time[i]);
        }

        printf("\nAverage Waiting Time: %.2f\n",
avg_waiting);
        printf("Average Turnaround Time: %.2f\n",
avg_turnaround);

        return 0;
    }
}
```

Output:

```
Enter total number of processes: 3
Enter time quantum: 4
Enter burst time for process P1: 10
Enter burst time for process P2: 4
Enter burst time for process P3: 5

Process    Burst Time    Waiting Time    Turnaround Time
P1         10           9              19
P2         4           4              8
P3         5           8              13

Average Waiting Time: 7.00
Average Turnaround Time: 13.33
```

RESULTS:

The Round Robin Scheduling Algorithm was successfully implemented. The program correctly calculated the **waiting time**, **turnaround time**, and their averages using a fixed **time quantum**.

CONCLUSION:

Round Robin is a **fair and efficient** scheduling technique used in time-sharing systems. This lab demonstrated how preemptive execution can be handled using time slices and how turnaround and waiting time are affected by time quantum.